TECHNICAL DOCUMENTATION

CRM Sales Data Pipeline

An ELT implementation with DuckDB, dbt and Apache Airflow

Prepared by: Saif Ramadan Mohamed Fathy
Institution : DEPI
Instructor: Kyrellos El Naggar
Source Code:
https://github.com/saif-ramadan/Airflow_Dbt_project.git
Date: October 2026

Table of Contents

1. Project Overview....4
1.1 Project Title....4
1.2 Executive Summary....4
1.3 Business Problem and Motivation....4
1.4 Project Objectives....5
1.5 Scope and Limitations....5
1.6 Business Value and Expected Outcomes....5
1.7 Evidence Base and Conventions....5
2. System Architecture....6
2.1 Overall Architecture....6
2.2 Roles of DuckDB, dbt and Apache Airflow in This Project....6
2.3 How Data Moves Through the Pipeline....6
2.4 End-to-End Execution....7
3. Database Design....8
3.1 Conceptual Schema....8
3.2 Logical Schema....9
3.3 Physical Schema (Designed)....10
3.4 As-Built Implementation in DuckDB....11
3.5 Design Versus Implementation....11
3.6 Constraints and Indexes Actually Present....12
4. Data Sources and Data Dictionary....13
4.1 Source Datasets....13
4.2 Data Dictionary....13
4.3 Data Profile and Limitations....14
5. dbt Transformation Layer....15
5.1 Project Structure....15
5.2 Configuration....15
5.3 Sources....15
5.4 Models....15
5.5 Lineage and Generated Documentation....16
5.6 Data Quality Tests....17
5.7 Test Execution Results....**Error! Bookmark not defined.**
6. Apache Airflow Orchestration....18
6.1 DAG Purpose and Workflow....18
6.2 DAG Configuration....18
6.3 Tasks....18
6.4 How Airflow Executes dbt....18
6.5 Failure Handling....19

6.6 Evidence of Successful Execution 19
7. End-to-End Data Pipeline 20
8. Testing and Validation 21
8.1 Implemented Tests 21
8.2 Pipeline Execution Validation 21
8.3 Database and Transformation Validation 21
8.4 Evidence Summary 21
9. Results and Discussion 22
9.1 What the Project Accomplishes 22
9.2 Final Outputs and Analytical Value 22
9.3 Benefits of the Architecture 22
9.4 Current Limitations 22
9.5 Potential Improvements (Proposed Future Work) 22
10. Conclusion 23

List of Figures

Figure 2.1: System architecture as implemented (derived from the DAG, scripts and dbt configuration)
Figure 3.1: Conceptual ER diagram from the repository (entity-relationship notation)
Figure 3.2: Conceptual dimensional sketch supplied separately (fact table with three dimensions)
Figure 3.3: Logical schema as supplied (keys only, with relationship lines)
Figure 3.4: Physical schema as supplied (designed tables, columns and data types)
Figure 3.5: As-built logical model derived from the dbt model SQL and tests (not from the supplied design images)
Figure 5.1: Model lineage redrawn from target/manifest.json (sources, staging, dimensions, fact and tests)
Figure 5.2: dbt documentation lineage graph (supplied screenshot, package crm_dbt)
Figure 5.3: dbt documentation, Database tab: 12 relations in crm_dw.main (supplied screenshot)
Figure 6.1: DAG workflow redrawn from dags/crm_pipeline_dag.py
Figure 6.2: Airflow graph view: all three tasks successful in a manual run (supplied screenshot)
Figure 7.1: End-to-end pipeline from CSV files to the star schema

List of Tables

Table 1.1: Project identifiers
Table 1.2: Scope of the implementation
Table 1.3: Evidence labels used in this report
Table 2.1: Component responsibilities
Table 3.1: Business entities (meaning taken from data_dictionary.csv; cardinalities from Figure 3.1)
Table 3.2: Keys shown in the logical schema (Figure 3.3)
Table 3.3: Designed physical schema (read from Figure 3.4)
Table 3.4: Relations in crm_dw.main
Table 3.5: Comparison of the designed and implemented schema
Table 4.1: Source files and raw tables (row counts computed from the CSV files)
Table 4.2: Data dictionary of the source tables
Table 4.3: Columns introduced by the dbt layer
Table 4.4: Source data profile (computed)
Table 5.1: dbt source declaration (models/staging/_sources.yml)
Table 5.2: Summary of dbt models (all materialized as views)
Table 5.3: Implemented dbt tests (models/marts/marts_models_test.yml) and results from run_results.json
Table 6.1: DAG settings
Table 6.2: Airflow tasks
Table 6.3: Execution evidence from Figure 6.2
Table 7.1: Pipeline stages
Table 8.1: Troubleshooting guide
Table 9.1: Evidence of successful execution
Table 9.2: Known issues and design observations (each is a finding about the current state, not a fix)
Table 9.3: Proposed additional tests

1. Project Overview

1.1 Project Title

CRM Sales Opportunities Data Pipeline: an ELT implementation with DuckDB, dbt and Apache Airflow.

Identifier	Value (as found in the repository)
Repository	`Airflow_Dbt_project` (GitHub: saif-ramadan/Airflow_Dbt_project)
Astro project name	`airflow-dbt-project` (`.astro/config.yaml`)
dbt project / profile	`crm_dbt` / `crm_dbt` (target `dev`)
Airflow DAG id	`crm_data_pipeline` (`dags/crm_pipeline_dag.py`)
DuckDB database	`crm_dw` (file `crm_dw.duckdb`, schema `main`)

Table 1.1: *Project identifiers*

1.2 Executive Summary

This project implements a daily-scheduled ELT pipeline for CRM sales-opportunity data. Four CSV files (85 accounts, 7 products, 35 sales agents and 8,800 sales opportunities, counted directly from the CSV files) are loaded as raw tables into a DuckDB database. Eight dbt models then transform them into four staging views and a star schema made of three dimension views and one fact view (`fct_sales_pipeline`). Five dbt data tests validate the fact table. Apache Airflow, running on the Astro Runtime image, orchestrates the process through a three-task DAG: `ingest_data >> dbt_run >> dbt_test`.

Screenshots of the Airflow UI show a successful manual run of the DAG on 2026-10-06 in which all three tasks succeeded, and the dbt documentation screenshots confirm the 12 relations in `crm_dw.main` and the lineage between them.

> **Key finding.** The conceptual, logical and physical schema diagrams describe a natural-key design (`account`, `product`, `sales_agent` as keys) with explicit VARCHAR/DECIMAL types and PK/FK constraints. The implemented dbt models use different key columns (`account_id`, `product_id`, `team_id`), are all materialized as views, and contain no constraint DDL. This report documents both and keeps them clearly separate (Sections 3.3 to 3.6).

1.3 Business Problem and Motivation

> **Inferred.** The repository contains no written problem statement. This section is derived from the data model and the supplied `data_dictionary.csv`. Replace or confirm it with your own project brief before submission.

The data describes a sales organisation that tracks each sales opportunity through the stages Prospecting, Engaging, and Won or Lost (as defined in `data_dictionary.csv`), and links every opportunity to a sales agent, a product and a customer account. In raw CSV form, answering questions such as revenue by product, win rate by sales team or pipeline value by account requires repeatedly joining four files by hand, with no automated checks and no repeatable refresh.

The motivation of the project is to replace that manual work with an automated, repeatable pipeline that produces a tested analytical model (a star schema) which can be queried directly.

1.4 Project Objectives

The objectives below are derived from what the implementation actually does:

- Ingest the four raw CSV files into DuckDB as raw tables on every run (full refresh).
- Standardise column names in a staging layer and build a dimensional model (three dimensions and one fact) with dbt.
- Validate the fact table's key and foreign-key columns with dbt data tests.
- Orchestrate ingestion, transformation and testing as one ordered, scheduled Airflow DAG with retries.
- Provide generated dbt documentation and lineage for the models.

1.5 Scope and Limitations

In scope (implemented)	Out of scope / not present in the repository
Batch ingestion from local CSV files DuckDB single-file warehouse 8 dbt models, 4 sources, 5 data tests One Airflow DAG with 3 BashOperator tasks Local development with the Astro CLI	BI or dashboard layer Incremental loading, snapshots or slowly changing dimensions CI/CD pipeline, alerting or monitoring integrations Cloud deployment Unit tests for the Python ingestion script

Table 1.2: *Scope of the implementation*

1.6 Business Value and Expected Outcomes

Demonstrated by the evidence: a repeatable, ordered pipeline that rebuilds the analytical model end to end, with automated referential and uniqueness checks on the fact table and a visible lineage graph.

Expected value (inferred): analysts can query one fact table and three dimensions rather than four raw files, which supports analyses such as pipeline value by stage, product, account or sales team. Section 9.5 lists data-quality gaps that currently limit the completeness of those analyses.

1.7 Evidence Base and Conventions

The GitHub repository could not be fetched directly (automated access is blocked), so the project was analysed from the uploaded archive `Airflow_Dbt_project.zip` and six uploaded images. The review was **static**: no code was executed, Docker was not started, and the DuckDB file was not opened. Statements are labelled as follows:

Label	Meaning
Verified	Directly visible in a repository file, a dbt artifact (`manifest.json`, `run_results.json`, `dbt.log`) or a supplied screenshot.
Computed	Calculated by the author of this report from the CSV files with a short Python script; not read from DuckDB.
Inferred	A reasonable conclusion that is not stated explicitly in the evidence. Needs your confirmation.
Proposed	A recommendation or future work. Not implemented.

Table 1.3: *Evidence labels used in this report*

2. System Architecture

2.1 Overall Architecture

The system is a batch ELT pipeline running inside an Astro Runtime container (`astrocrpublic.azurecr.io/runtime:3.3-8`, from the `Dockerfile`). Airflow runs three shell commands in sequence. The first loads raw tables into DuckDB with a Python script; the other two run `dbt run` and `dbt test` against the same DuckDB file.

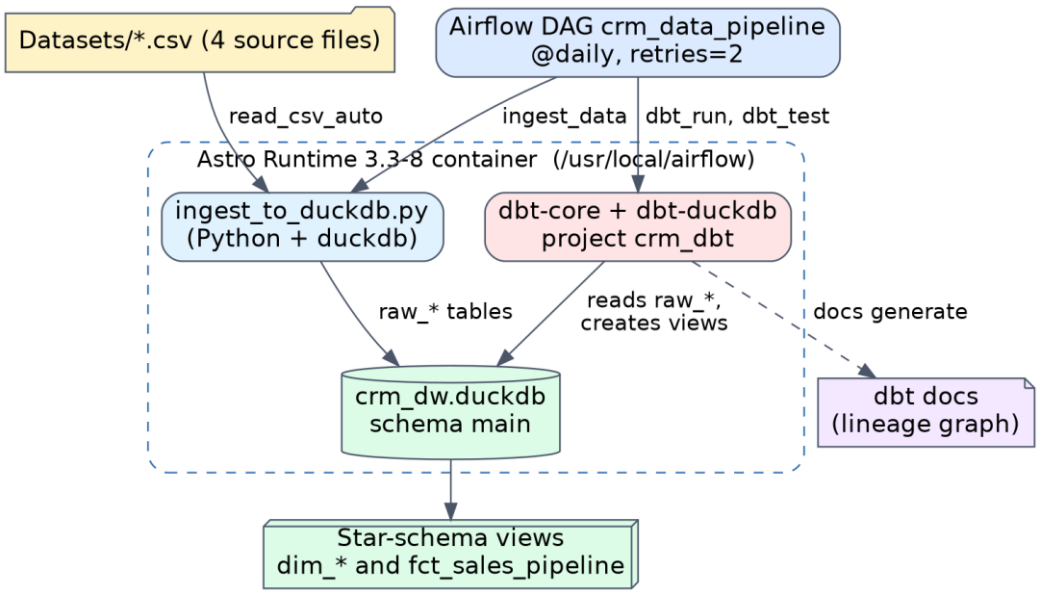


Figure 2.1: *System architecture as implemented (derived from the DAG, scripts and dbt configuration)*

2.2 Roles of DuckDB, dbt and Apache Airflow in This Project

Component	Actual role in this implementation	Evidence
DuckDB	Embedded analytical database stored in a single file, `crm_dw.duckdb`. Holds the four raw tables created by the ingestion script and the eight dbt views, all in database `crm_dw`, schema `main`.	`ingest_to_duckdb.py`; `profiles.yml`; dbt docs screenshot
dbt (dbt-core + dbt-duckdb)	Transformation and testing layer. Reads the raw tables as the source `raw_crm`, builds staging and mart models as views, runs five data tests, and generates documentation and lineage. Manifest reports dbt 1.12.5 with the duckdb adapter.	`models/`; `manifest.json`; `run_results.json`
Apache Airflow	Orchestrator. One DAG (`crm_data_pipeline`) with three `BashOperator` tasks enforces order, schedule (`@daily`) and retries (2). It does not transform data itself.	`crm_pipeline_dag.py`; Airflow screenshot
Python (duckdb package)	Ingestion: reads each CSV with `read_csv_auto` and writes it to a raw table with `CREATE OR REPLACE TABLE`.	`ingest_to_duckdb.py`
Astro CLI / Astro Runtime	Provides the containerised Airflow environment for local development.	`Dockerfile`; `.astro/config.yaml`

Table 2.1: *Component responsibilities*

2.3 How Data Moves Through the Pipeline

1. CSV files in `Datasets/` are read by `ingest_to_duckdb.py` using DuckDB's `read_csv_auto`.
2. Each file is written to a raw table (`raw_accounts`, `raw_products`, `raw_sales_pipeline`, `raw_sales_teams`) with `CREATE OR REPLACE TABLE`, so every run fully replaces the raw data.

3. dbt declares these tables as the source `raw_crm` (schema `main`) and builds four staging views that rename columns.
4. dbt builds three dimension views from the staging views, adding surrogate keys for products and sales teams.
5. dbt builds `fct_sales_pipeline` by left-joining the staged pipeline to the three dimensions.
6. `dbt test` runs five data tests on the fact view. A failing test makes the Airflow task fail.

2.4 End-to-End Execution

When the DAG runs, the Airflow scheduler starts `ingest_data`; on success it starts `dbt_run`; on success it starts `dbt_test`. Each task is a shell command executed inside the container. dbt finds its connection settings through `--profiles-dir .`, which points to `crm_dbt/profiles.yml`. That file and the ingestion script both resolve to the same database path, `/usr/local/airflow/crm_dw.duckdb`, which is what connects the three tasks.

> **Inferred.** In a standard Astro project the `Dockerfile` copies project files into the image at build time, so the DuckDB file written inside the container is separate from the `crm_dw.duckdb` file in your project folder (which was produced by local runs). Confirm this with the commands in Section 8.7 before describing persistence in your submission.

3. Database Design

This chapter documents the database at three design levels using the supplied schema images, and then documents what is actually implemented in DuckDB (Sections 3.4 to 3.6). Design diagrams are reproduced as supplied; they have not been altered.

3.1 Conceptual Schema

Two conceptual diagrams were provided. The ER diagram stored in the repository (`schema/Conceptual Schema.png`) shows entities, attributes and cardinalities. The second, supplied separately, is a dimensional sketch with a fact table and three dimensions.

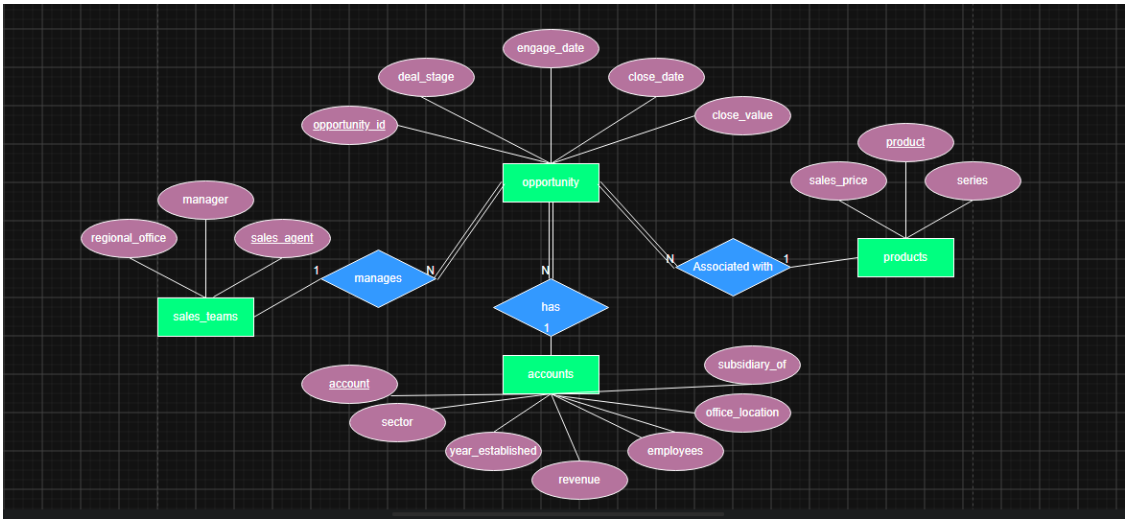


Figure 3.1: *Conceptual ER diagram from the repository (entity-relationship notation)*

Reading of Figure 3.1 (verified from the image): four entities (`opportunity`, `sales_teams`, `accounts`, `products`) and three relationships, each one-to-many from the dimension-like entity to `opportunity`: `sales_teams` manages many opportunities, `accounts` has many opportunities, and `products` is associated with many opportunities. Underlined attributes mark identifiers: `opportunity_id`, `sales_agent`, `account` and `product`.

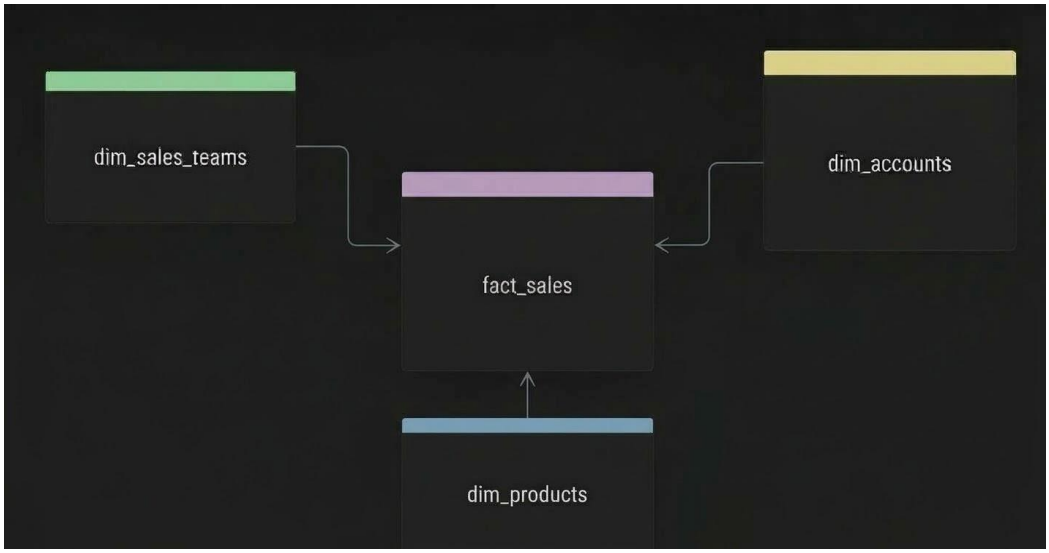


Figure 3.2: *Conceptual dimensional sketch supplied separately (fact table with three dimensions)*

Entity	Business meaning	Relationship to opportunity
opportunity (fact_sales / fact_sales_pipeline / fct_sales_pipeline)	A single sales opportunity moving through the stages Prospecting, Engaging and Won or Lost; carries dates and the deal value.	Central entity
sales_teams	A sales agent, their manager and regional office.	1 : N (manages)
accounts	A customer company with sector, size, location and parent company.	1 : N (has)
products	A product with its series and suggested retail price.	1 : N (associated with)

Table 3.1: *Business entities (meaning taken from data_dictionary.csv; cardinalities from Figure 3.1)*

> **Naming inconsistency.** The central entity is called `opportunity` in Figure 3.1, `fact_sales` in Figure 3.2, `fact_sales_pipeline` in the logical and physical diagrams, and `fct_sales_pipeline` in the dbt project and documentation.

3.2 Logical Schema

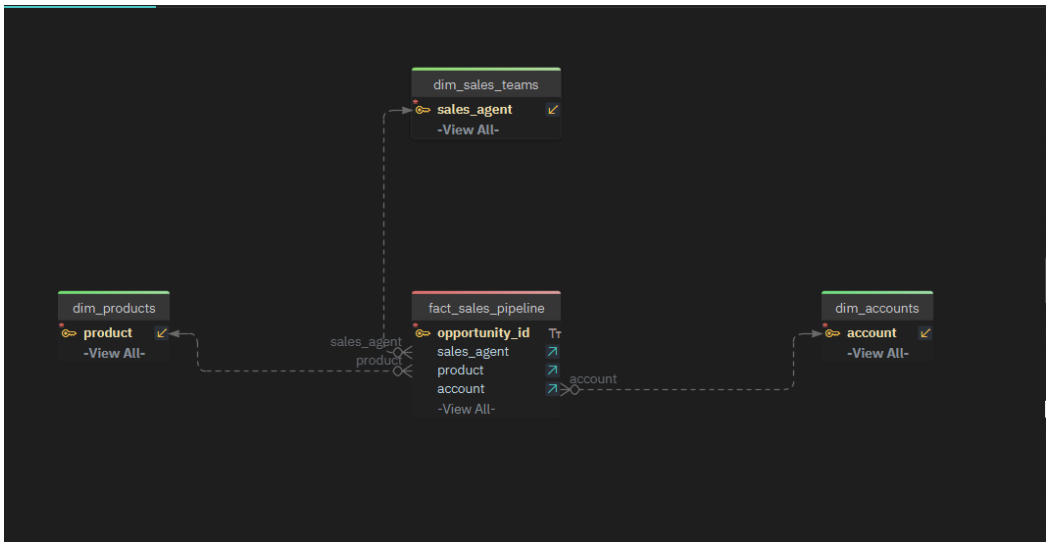


Figure 3.3: *Logical schema as supplied (keys only, with relationship lines)*

Table	Primary key	Foreign keys	Related to
fact_sales_pipeline	opportunity_id	sales_agent, product, account	dim_sales_teams, dim_products, dim_accounts
dim_sales_teams	sales_agent	-	-
dim_products	Product	-	-
dim_accounts	Account	-	-

Table 3.2: *Keys shown in the logical schema (Figure 3.3)*

- **Cardinality.** The many-side (crow's foot with optional marker) is drawn at the fact table end of each dashed relationship line. The marker at the dimension end is not legible in the image, so it is **not verified**; the one-to-many reading comes from Figure 3.1.
- **Normalisation decisions (inferred).** The design is a star schema: dimensions are denormalised descriptive tables, and the fact table holds the measure (`close_value`), dates, stage and the degenerate identifier `opportunity_id`. The parent-company relationship of an account is kept as a text attribute (`subsidiary_of`) with no relationship line, so it is not modelled as a self-reference.

3.3 Physical Schema (Designed)

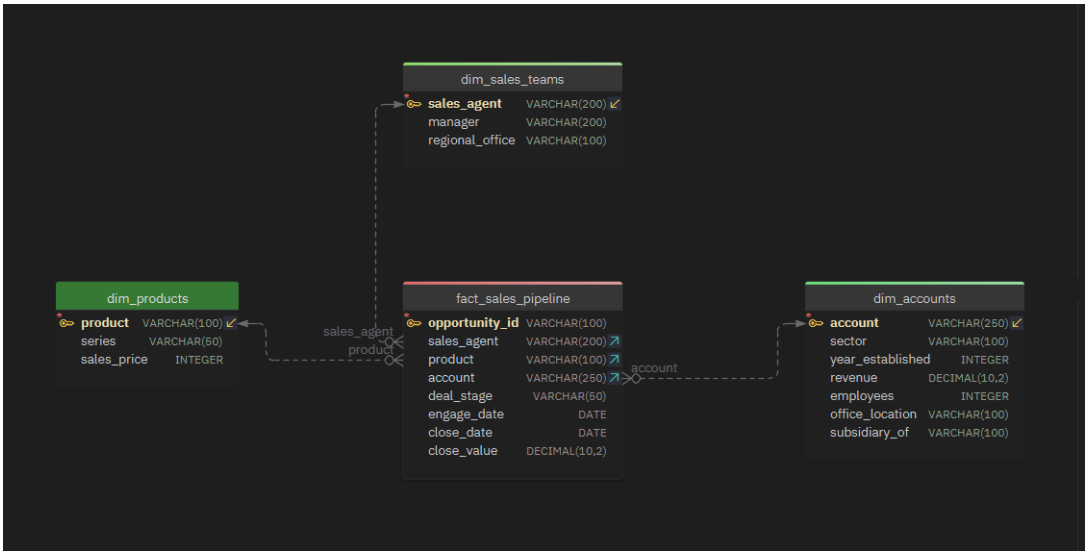


Figure 3.4: *Physical schema as supplied (designed tables, columns and data types)*

Table	Column	Designed type	Key
dim_sales_teams	sales_agent	VARCHAR(200)	PK
	Manager	VARCHAR(200)	
	regional_office	VARCHAR(100)	
dim_products	Product	VARCHAR(100)	PK
	Series	VARCHAR(50)	
	sales_price	INTEGER	
dim_accounts	Account	VARCHAR(250)	PK
	Sector	VARCHAR(100)	
	year_established	INTEGER	
	Revenue	DECIMAL(10,2)	
	Employees	INTEGER	
	office_location	VARCHAR(100)	
	subsidiary_of	VARCHAR(100)	
fact_sales_pipeline	opportunity_id	VARCHAR(100)	PK
	sales_agent	VARCHAR(200)	FK
	Product	VARCHAR(100)	FK
	Account	VARCHAR(250)	FK
	deal_stage	VARCHAR(50)	
	engage_date	DATE	
	close_date	DATE	
	close_value	DECIMAL(10,2)	

Table 3.3: *Designed physical schema (read from Figure 3.4)*

> **This is a design artifact, not the deployed schema.** No `CREATE TABLE` statement, constraint or index definition exists in the repository, so the types and keys above are not implemented in DuckDB (see Sections 3.4 to 3.6).

3.4 As-Built Implementation in DuckDB

The following objects exist in database `crm_dw`, schema `main` (12 relations, confirmed by the dbt documentation screenshot, Figure 5.3, and by `manifest.json`):

Object	Type	Created by	Notes
raw_accounts, raw_products, raw_sales_teams, raw_sales_pipeline	Table	`ingest_to_duckdb.py` (`CREATE OR REPLACE TABLE ... AS SELECT * FROM read_csv_auto(...)`)	Column types are inferred by DuckDB from the CSV files. They were not read from the database file.
stg_accounts, stg_products, stg_sales_teams, stg_sales_pipeline	View	dbt (`models/staging`)	Column renames only.
dim_accounts, dim_products, dim_sales_teams	View	dbt (`models/marts`)	`dim_products.product_id` and `dim_sales_teams.team_id` are generated with `row_number() over ()`.
fct_sales_pipeline	View	dbt (`models/marts`)	Left joins from staged pipeline to the three dimensions.

Table 3.4: *Relations in crm_dw.main*

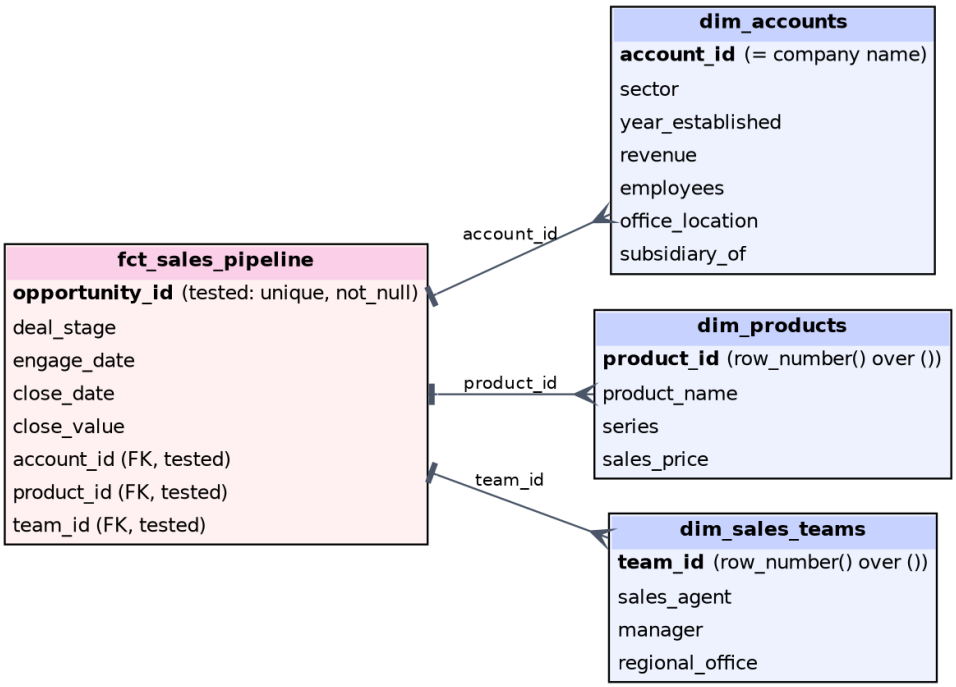


Figure 3.5: *As-built logical model derived from the dbt model SQL and tests (not from the supplied design images)*

3.5 Design Versus Implementation

Aspect	Designed (Figures 3.3 and 3.4)	Implemented (dbt models)
Fact table name	`fact_sales_pipeline`	`fct_sales_pipeline`
Account key	`account` (company name), PK of `dim_accounts`	`account_id` (same company-name values, renamed in staging)

Aspect	Designed (Figures 3.3 and 3.4)	Implemented (dbt models)
Product key	`product` (name), PK of `dim_products`	`product_id`, a surrogate integer from `row_number() over ()`; name kept as `product_name`
Sales team key	`sales_agent` (name), PK of `dim_sales_teams`	`team_id`, a surrogate integer from `row_number() over ()`; `sales_agent` kept as a descriptive column
Fact foreign keys	`sales_agent`, `product`, `account` (text)	`team_id`, `product_id`, `account_id`
Data types	Explicit VARCHAR(n), INTEGER, DECIMAL(10,2), DATE	No explicit types; raw types inferred by `read_csv_auto`, views inherit them
Constraints	PK and FK shown	None in DuckDB; integrity is checked only by dbt tests
Object type	Tables	Views (dbt default)

Table 3.5: *Comparison of the designed and implemented schema*

3.6 Constraints and Indexes Actually Present

- **Primary keys, foreign keys, unique constraints, check constraints and indexes:** none found anywhere in the repository (**verified**). The dbt models are views, and DuckDB views cannot carry table constraints.
- **Integrity checking:** performed by five dbt tests on `fct_sales_pipeline` (Section 5.6).
- **Proposed:** if enforced constraints are required, materialise the marts as tables with dbt model contracts, or add explicit DDL. This is future work and is not implemented.

4. Data Sources and Data Dictionary

4.1 Source Datasets

The pipeline reads four CSV files from the `Datasets/` folder. A fifth file, `data_dictionary.csv`, documents their fields and is not loaded into the database.

CSV file	Raw table	Rows (excl. header)	Columns
accounts.csv	raw_accounts	85	7
products.csv	raw_products	7	3
sales_teams.csv	raw_sales_teams	35	3
sales_pipeline.csv	raw_sales_pipeline	8,800	8

Table 4.1: *Source files and raw tables (row counts computed from the CSV files)*

Origin. The repository does not document where the data comes from. The files are dated 2024-04-03 and their structure is consistent with a publicly available CRM sales-opportunities sample dataset. This is an **inference**; confirm the exact source and licence and cite it in your submission.

4.2 Data Dictionary

Descriptions are taken from `Datasets/data_dictionary.csv`. Designed types and key status come from the physical schema diagram (Figure 3.4) and are design-level only (Section 3.3).

Table	Column	Description (data_dictionary.csv)	Designed type	Key (design)
accounts	account	Company name	VARCHAR(250)	PK
accounts	sector	Industry	VARCHAR(100)	
accounts	year_established	Year Established	INTEGER	
accounts	revenue	Annual revenue (in millions of USD)	DECIMAL(10,2)	
accounts	employees	Number of employees	INTEGER	
accounts	office_location	Headquarters	VARCHAR(100)	
accounts	subsidiary_of	Parent company	VARCHAR(100)	
products	product	Product name	VARCHAR(100)	PK
products	series	Product series	VARCHAR(50)	
products	sales_price	Suggested retail price	INTEGER	
sales_teams	sales_agent	Sales agent	VARCHAR(200)	PK
sales_teams	manager	Respective sales manager	VARCHAR(200)	
sales_teams	regional_office	Regional office	VARCHAR(100)	
sales_pipeline	opportunity_id	Unique identifier	VARCHAR(100)	PK
sales_pipeline	sales_agent	Sales agent	VARCHAR(200)	FK
sales_pipeline	product	Product name	VARCHAR(100)	FK
sales_pipeline	account	Company name	VARCHAR(250)	FK
sales_pipeline	deal_stage	Sales pipeline stage (Prospecting > Engaging > Won / Lost)	VARCHAR(50)	

Table	Column	Description (data_dictionary.csv)	Designed type	Key (design)
sales_pipeline	engage_date	Date in which the "Engaging" deal stage was initiated	DATE	
sales_pipeline	close_date	Date in which the deal was "Won" or "Lost"	DATE	
sales_pipeline	close_value	Revenue from the deal	DECIMAL(10,2)	

Table 4.2: *Data dictionary of the source tables*

Columns added by dbt (implemented, not in the source files):

Model	Column	Meaning	How it is produced
stg_accounts, dim_accounts, fct_sales_pipeline	account_id	Company name used as the account key	`account` renamed in staging
stg_products, dim_products	product_name	Product name	`product` renamed in staging
dim_products, fct_sales_pipeline	product_id	Surrogate product key	`row_number() over ()` in `dim_products`; joined by product name in the fact
dim_sales_teams, fct_sales_pipeline	team_id	Surrogate sales-team key	`row_number() over ()` in `dim_sales_teams`; joined by `sales_agent` in the fact

Table 4.3: *Columns introduced by the dbt layer*

4.3 Data Profile and Limitations

The figures below were **computed** from the CSV files (not from DuckDB) and describe the source data as ingested.

Observation	Value	Consequence
Opportunities (distinct `opportunity_id`)	8,800 of 8,800	The identifier is unique in the source.
Deal stages	Won 4,238; Lost 2,473; Engaging 1,589; Prospecting 500	Open deals (Engaging, Prospecting) are 2,089 of 8,800.
Empty `close_date` and `close_value`	2,089 each	Matches the number of open deals.
Empty `engage_date`	500	Matches the Prospecting deals.
Empty `account` in the pipeline	1,425 (16.2%), all in Engaging or Prospecting	These opportunities get a NULL `account_id` in the fact view.
Product value `GTXPro` in the pipeline	1,480 rows (16.8%)	`products.csv` contains `GTX Pro`, so the join on product name finds no match and `product_id` is NULL.
Sales agents missing from `sales_teams`	0	All agents match.
Spelling `technolgy` in `sector`	12 of 85 accounts	Passed through unchanged; affects grouping by sector.
Accounts with a parent company	15 of 85	Parent is stored as text only.

Table 4.4: *Source data profile (computed)*

- **Assumption:** `revenue` is expressed in millions of USD, as stated in the dictionary.
- **Limitation:** the data is a static snapshot loaded by full refresh; there is no incremental or historical tracking.

5. dbt Transformation Layer

5.1 Project Structure

```
crm_dbt/
  dbt_project.yml
  profiles.yml
  models/
    staging/
      _sources.yml
      stg_accounts.sql  stg_products.sql
      stg_sales_pipeline.sql  stg_sales_teams.sql
    marts/
      dim_accounts.sql  dim_products.sql  dim_sales_teams.sql
      fct_sales_pipeline.sql
      marts_models_test.yml
  analyses/ macros/ seeds/ snapshots/ tests/   (contain only .gitkeep)
  target/  logs/                               (generated by dbt)
  dev.duckdb                                   (12 KB, not referenced by profiles.yml)
```

dbt-core 1.12.5 with the `duckdb` adapter is reported by `manifest.json`. The profile `crm_dbt` has one output, `dev`, of type `duckdb`, with the path `/usr/local/airflow/crm_dw.duckdb`.

5.2 Configuration

`dbt_project.yml` sets the standard directories and one model configuration:

```
models:
  crm_dbt:
    example:
      +materialized: view
```

There is no `models/example` folder, so this setting matches no model and has no effect. All eight models use dbt's default materialization, **view**, which `manifest.json` confirms for every model.

5.3 Sources

Source name	Schema	Tables	Source tests / freshness
raw_crm	main	raw_accounts, raw_products, raw_sales_pipeline, raw_sales_teams	None defined

Table 5.1: *dbt source declaration (models/staging/_sources.yml)*

5.4 Models

Model	Layer	Inputs	Transformation logic	Output columns
stg_accounts	Staging	source raw_accounts	Renames `account` to `account_id`; passes all other columns.	account_id, sector, year_established, revenue, employees, office_location, subsidiary_of
stg_products	Staging	source raw_products	Renames `product` to `product_name`.	series, product_name, sales_price
stg_sales_teams	Staging	source raw_sales_teams	Selects the three columns without renaming.	sales_agent, manager, regional_office
stg_sales_pipeline	Staging	source raw_sales_pipeline	Renames `product` to `product_name` and `account` to `account_id`.	opportunity_id, sales_agent, product_name, account_id, deal_stage, engage_date, close_date, close_value

Model	Layer	Inputs	Transformation logic	Output columns
dim_accounts	Mart (dimension)	stg_accounts	Pass-through of the staged columns.	account_id, sector, year_established, revenue, employees, office_location, subsidiary_of
dim_products	Mart (dimension)	stg_products	Adds `product_id = row_number() over ()`.	product_id, product_name, series, sales_price
dim_sales_teams	Mart (dimension)	stg_sales_teams	Adds `team_id = row_number() over ()`.	team_id, sales_agent, manager, regional_office
fct_sales_pipeline	Mart (fact)	stg_sales_pipeline, dim_accounts, dim_products, dim_sales_teams	Left joins the pipeline to accounts on `account_id`, to products on `product_name` and to teams on `sales_agent`. Keeps the five pipeline attributes and the three resolved keys.	opportunity_id, deal_stage, engage_date, close_date, close_value, account_id, product_id, team_id

Table 5.2: *Summary of dbt models (all materialized as views)*

Business rules visible in the code. Left joins preserve every opportunity even when no dimension row matches, so unmatched opportunities appear with NULL keys. The fact keeps only keys, not the agent or product text, so descriptive attributes are reached through the dimensions. The layer classification (staging and marts) follows the folder names in the repository.

`dim_products.product_id` and `dim_sales_teams.team_id` come from `row_number() over ()` with no `ORDER BY`. Because the dimensions are views, the numbering is recomputed whenever they are queried and is not guaranteed to be stable across runs. This is a design risk (Section 9.5), not an observed failure.

5.5 Lineage and Generated Documentation

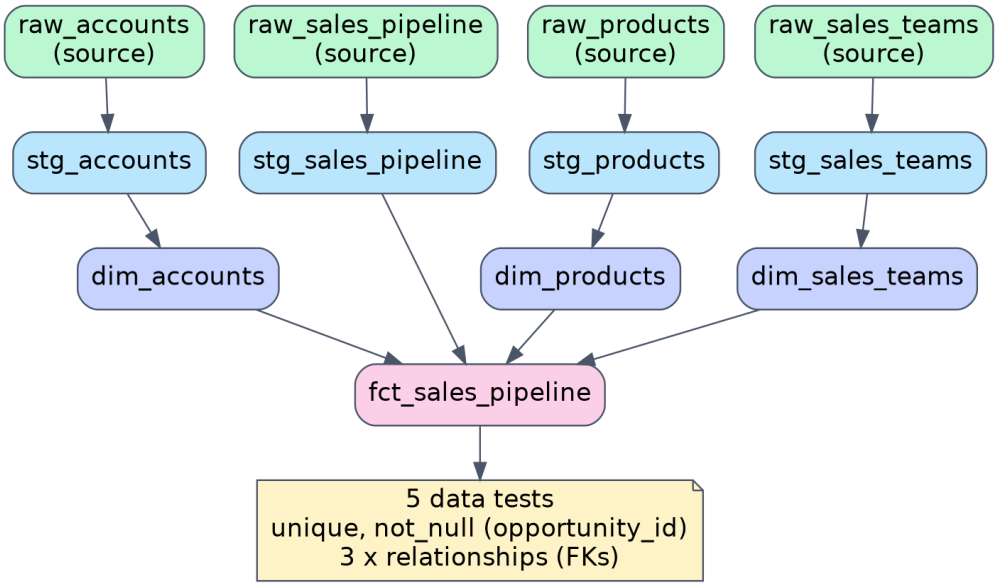


Figure 5.1: *Model lineage redrawn from target/manifest.json (sources, staging, dimensions, fact and tests)*



Figure 5.2: *dbt documentation lineage graph (supplied screenshot, package crm_dbt)*

The screenshot agrees with the manifest: four sources feed four staging models, three of which pass through a dimension before the fact, while `stg_sales_pipeline` feeds the fact directly. The five tests exist in the manifest but are not drawn in the screenshot.

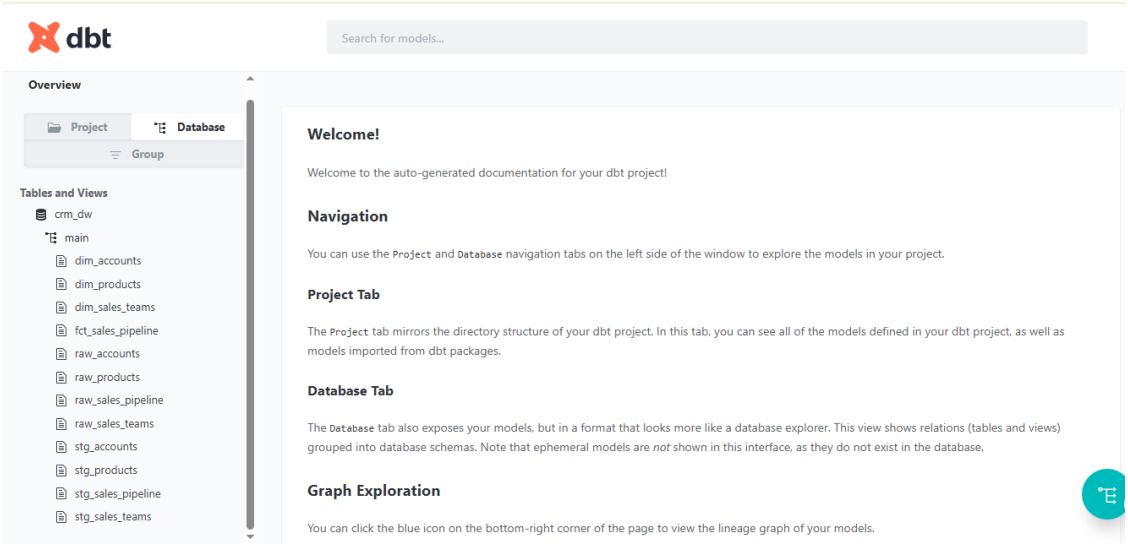


Figure 5.3: *dbt documentation, Database tab: 12 relations in crm_dw.main (supplied screenshot)*

`catalog.json` is not in the archive, and only `fct_sales_pipeline` has a model description (written in Arabic). No column descriptions are defined, so the generated documentation shows names and types without business meaning.

5.6 Data Quality Tests

Test (generic)	Column	What it checks	Result
unique	fct_sales_pipeline.opportunity_id	No duplicate opportunity identifiers	Pass
not_null	fct_sales_pipeline.opportunity_id	Every row has an identifier	Pass
relationships	account_id to dim_accounts.account_id	Non-null account keys exist in the dimension	Pass
relationships	product_id to dim_products.product_id	Non-null product keys exist in the dimension	Pass
relationships	team_id to dim_sales_teams.team_id	Non-null team keys exist in the dimension	Pass

6. Apache Airflow Orchestration

6.1 DAG Purpose and Workflow

The DAG `crm_data_pipeline` (description in code: "Full End-to-End Pipeline project") runs the whole pipeline in order: load raw data, build the dbt models, then test them. The runtime is `astrocrpublic.azurecr.io/runtime:3.3-8` and the UI in the screenshot shows a *Dag Version* field and an *Assets* menu, which suggests the Airflow 3 line (**inferred**).

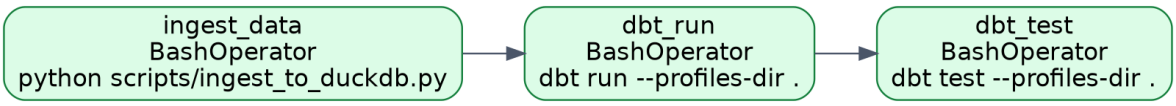


Figure 6.1: *DAG workflow redrawn from dags/crm_pipeline_dag.py*

6.2 DAG Configuration

Setting	Value in code	Comment
dag_id	`crm_data_pipeline`	
owner	`Seif`	In `default_args`
schedule	`@daily`	Runs once per day
start_date	`datetime(2005, 4, 24)`	Unusual date; harmless with `catchup=False`, but likely a placeholder
catchup	`False`	No backfill of past intervals
retries / retry_delay	2 / 1 minute	Applies to every task
tags, timeouts, SLAs, callbacks	Not set	No alerting on failure
Time zone	Africa/Cairo	Set in `.env` (default and UI time zone)

Table 6.1: *DAG settings*

6.3 Tasks

Task id	Operator	Command	Depends on	Purpose
ingest_data	BashOperator	`python /usr/local/airflow/scripts/ingest_to_duckdb.py`	None	Load the four CSV files into raw tables
dbt_run	BashOperator	`cd /usr/local/airflow/crm_dbt && dbt run --profiles-dir .`	ingest_data	Build the 8 dbt models
dbt_test	BashOperator	`cd /usr/local/airflow/crm_dbt && dbt test --profiles-dir .`	dbt_run	Run the 5 data tests

Table 6.2: *Airflow tasks*

Dependencies are declared as `ingest_data >> dbt_run >> dbt_test`.

6.4 How Airflow Executes dbt

dbt is not called through a dedicated provider. Each `BashOperator` runs a shell command inside the Airflow container, and Airflow marks the task failed when the command exits with a non-zero status. `dbt run` and `dbt test` both execute the whole project (no selectors). If any dbt test fails, `dbt test` exits with an error and the `dbt_test` task fails.

6.5 Failure Handling

- **Implemented:** two retries with a one-minute delay for every task, and the default trigger rule, so a task starts only if its upstream task succeeded.
- **Not implemented:** failure notifications, SLAs, task timeouts or custom callbacks.
- **Gap in the ingestion script:** when a CSV file is missing, `ingest_to_duckdb.py` prints a message and continues without raising an error, so `ingest_data` can succeed while a raw table is stale or absent.

6.6 Evidence of Successful Execution

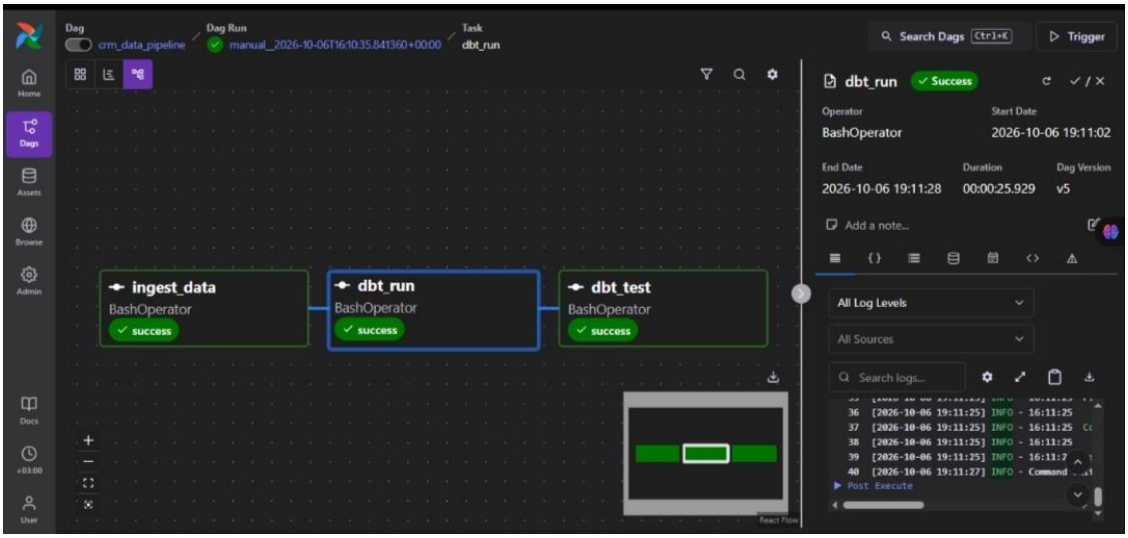


Figure 6.2: *Airflow graph view: all three tasks successful in a manual run (supplied screenshot)*

Item	Value shown in the screenshot
DAG run	`manual__2026-10-06T16:10:35.841360+00:00` (manually triggered)
Task states	`ingest_data`, `dbt_run`, `dbt_test`: success
`dbt_run` timing	Start 2026-10-06 19:11:02, end 19:11:28 (UI time, +03:00), duration 00:00:25.929
DAG version	v5

Table 6.3: *Execution evidence from Figure 6.2*

The screenshot proves one successful manual run. It does not show a scheduled run, the output of `ingest_data`, or the test counts of `dbt_test`.

7. End-to-End Data Pipeline

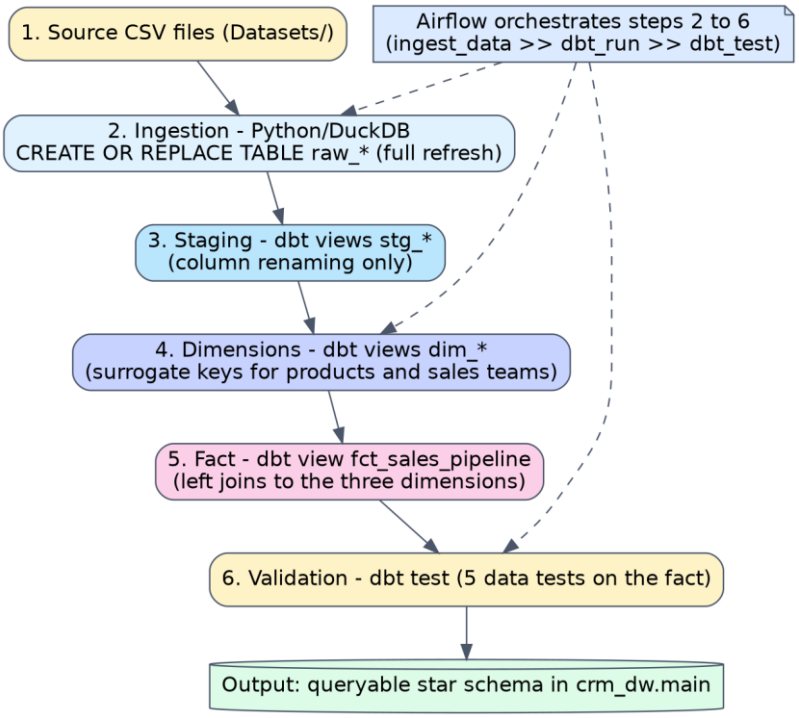


Figure 7.1: *End-to-end pipeline from CSV files to the star schema*

#	Stage	Input	Operation	Technology	Output	Depends on	Validation
1	Ingest	4 CSV files	`read_csv_auto`, `CREATE OR REPLACE TABLE`	Python, DuckDB	4 raw tables	CSV files present	Row count printed; missing file only logged
2	Stage	raw_* tables	Rename columns	dbt views	4 stg_* views	Stage 1	None
3	Dimensions	stg_* views	Pass through; add surrogate keys	dbt views	3 dim_* views	Stage 2	None directly
4	Fact	stg_sales_pipeline, dim_*	Left joins	dbt view	fct_sales_pipeline	Stages 2, 3	Tests in stage 5
5	Test	fct_sales_pipeline, dim_*	5 data tests	dbt test	Pass or fail	Stage 4	unique, not_null, 3 relationships

Table 7.1: *Pipeline stages*

Execution flow:

1. The scheduler creates a DAG run (daily or manual trigger).
2. `ingest_data` loads the raw tables, replacing previous content.
3. `dbt_run` creates or replaces the 8 views in dependency order.
4. `dbt_test` runs the 5 tests; success finishes the run.
5. Failures are retried twice at one-minute intervals before the task is marked failed.

8. Testing and Validation

8.1 Implemented Tests

Five dbt data tests on `fct_sales_pipeline` (Table 5.3). The repository's `tests/` folder in `crm_dbt` is empty, and the Astro template test `tests/dags/test_dag_example.py` exists but is not adapted to this DAG: it expects every DAG to have tags, and `crm_data_pipeline` has none. It was not executed.

8.2 Pipeline Execution Validation

- Airflow screenshot: one manual run with all three tasks successful (Section 6.6).
- dbt log: successful `dbt run` of 8 models and `dbt test` of 5 tests from a local run.

8.3 Database and Transformation Validation

The DuckDB file was not queried in this review. **Proposed** reconciliation queries to run and record as evidence:

```
select count(*) from raw_sales_pipeline;       -- expected 8800
select count(*) from fct_sales_pipeline;       -- expected 8800
select count(*) from fct_sales_pipeline where product_id is null;  -- source data suggests 1480
select count(*) from fct_sales_pipeline where account_id is null;  -- source data suggests 1425
describe raw_sales_pipeline;                    -- actual inferred column types
```

8.4 Evidence Summary

Evidence	What it shows	Source
dbt run, 8 of 8 models OK	Models build against DuckDB	`logs/dbt.log`
dbt test, 5 of 5 pass	Key and relationship checks pass	`run_results.json`
Airflow run, 3 of 3 tasks success	Pipeline runs end to end in Airflow	Screenshot
12 relations in crm_dw.main	Raw, staging and mart objects exist	dbt docs screenshot

Table 9.1: *Evidence of successful execution*

9. Results and Discussion

9.1 What the Project Accomplishes

- An automated pipeline that loads four CSV files, builds a star schema of eight dbt models and checks it with five tests, orchestrated by one Airflow DAG (**verified**).
- A successful end-to-end Airflow run, with the dbt run completing in about 26 seconds (**verified** from the screenshot).
- Generated dbt documentation with a lineage graph (**verified** from screenshots).

9.2 Final Outputs and Analytical Value

The final outputs are the views `dim_accounts`, `dim_products`, `dim_sales_teams` and `fct_sales_pipeline`. They let an analyst query opportunity values, stages and dates by account, product and sales team without re-joining raw files (**inferred** analytical use). Results for the 1,480 `GTXPro` opportunities cannot currently be attributed to a product through the dimension.

9.3 Benefits of the Architecture

- DuckDB needs no database server, so the whole stack runs on a laptop.
- dbt makes transformations declarative, dependency-aware, testable and documented.
- Airflow adds scheduling, ordering, retries and a visual run history.

9.4 Current Limitations

See Section 9.5. The main limitations are the design-versus-implementation gap, views without constraints, silent NULL keys, limited test coverage and minimal failure handling.

9.5 Potential Improvements (Proposed Future Work)

- Standardise product names in staging (for example mapping `GTXPro` to `GTX Pro`) and decide how to treat opportunities without an account.
- Materialise the marts as tables, define model contracts with constraints, and use a deterministic surrogate key (an ordered key or a hash of the natural key).
- Align the schema diagrams, model names and keys, and add column descriptions in the dbt YAML.
- Add the tests in Table 9.3, task timeouts, failure alerts, DAG tags and the current Airflow import path.
- Add CI that runs `dbt build`, and consider incremental loading for the fact table.

10. Conclusion

The project implements a complete, orchestrated ELT workflow on a lightweight stack: Python and DuckDB for ingestion and storage, dbt for transformation, modelling and testing, and Airflow on Astro Runtime for scheduling and dependency management. The evidence supports a working pipeline of 4 sources, 8 models and 5 tests with a successful Airflow run.

It demonstrates ELT design, dimensional modelling, declarative transformations, data testing and workflow orchestration. The principal remaining work is to align the designed schema with the implemented one, tighten data-quality checks so that NULL keys are caught, and complete the operational hardening listed in Section 10.5.